Methodology

Modeling Framework

The robot statics are modeled using SoRoSim, a MATLAB-based toolbox implementing the Geometric Variable-Strain (GVS) formulation of Cosserat rod theory for continuum robots. This framework was selected for its computational efficiency and native support for tendon-driven actuation.

Robot Manufacturing

We selected a 200 mm backbone length to balance adequate reach with manufacturability on standard FDM 3D printer beds. We chose Bambu's PLA Basic as our printing material with the following material properties referenced from their datasheet (Young's modulus E = 2.58 GPa, Poisson's ratio v = 0.34). Robot designs were prototyped in SolidWorks and printed on the Bambu X1 Carbon for a precise physical model.

Robot Model Configuration

We model our robot as a fixed cantilever beam initially orientated along the x-axis. The distal end is unconstrained with two degrees of freedom, bending about the y-axis and z-axis. The proximal end is fixed to the global coordinate frame origin.

To quantify reaction forces and moments at the base, a six-degree-of-freedom constraint (three translational, three rotational) is enforced at the proximal end. This constraint is position-controlled, fixed at a specified position and orientation by the user. SoRoSim then allows us to solve the six-dimensional wrench required to maintain this specified fixed configuration.

To create different designs of beams with different cross-sectional areas, we utilized attributes of the SorosimLink class. This class allows the user to create a rigid or soft body which can later be attached via joints to other links to form a manipulator. Link divisions allow you to split the link into discretized intervals, each interval with its own properties.

We compare three backbone designs: cylindrical, tapered, and optimized. The cylindrical and tapered designs use a single division whose radius is defined by a linear function $r(s) = as + b$, where s is the normalized distance along the backbone (0 to 1). By varying parameters a and b, we can create either constant-radius (cylindrical) or tapered profiles.

For the optimized design, we discretized the backbone into 20 equally spaced divisions. This number balances geometric resolution with computational efficiency, as more segments increase simulation time. Each segment has an independent constant radius,

enabling arbitrary cross-sectional profiles along the backbone length. As we will discuss later, we plan to let an optimizer select these 20 radii to minimize force estimation error.

Tendon Actuation Model

Three tendons are routed along straight, constant-radius paths at 120° circumferential intervals with an 8 mm radial offset from the backbone centerline. Each tendon path is modeled as a constraint curve rather than through explicit routing disk geometry, a technique demonstrated in [1] and [2]. This simplification assumes the physical routing disks are sufficiently thin that their bending stiffness contribution is negligible compared to the backbone, reducing model complexity while maintaining computational efficiency.

Each tendon is independently actuated via tension control $[\tau_1, \tau_2, \tau_3]$. Tensioning a tendon pulls the robot backbone toward the sensor, generating a wrench. Since we aim to measure only wrenches from external forces, we needed to decouple wrenches caused by tendon actuation alone. To achieve this, we mounted the tendon actuation units proximal to the 6-axis force-torque sensor. This design ensures that tendon forces remain internal to the system, preventing the sensor from directly measuring wrenches due to tendon actuation in isolation. However, the tendons still indirectly affect measurements: by pulling a tendon, the backbone's equilibrium configuration changes, which alters the mapping between applied external loads and measured base reactions.

Applied Point Forces

While calculating the proximal wrench from an applied force is straightforward, the inverse problem is often ill-posed. Multiple distinct point forces can produce identical or nearly identical proximal wrenches, making them difficult to distinguish—particularly when sensor noise and resolution limits further reduce distinguishability [3]. This ill-conditioning also degrades the convergence speed of force estimates.

Parameterizing applied loads as Gaussian distributions mitigates these issues. Studies have shown this approach improves both convergence speed and accuracy while enhancing robustness to measurement noise [3]. We therefore implement Gaussian load parameterization in this work.

Applied forces are characterized by four components:

- F: force magnitude
- s: position along the total arc length relative to the origin
- θ_1: roll angle about x-axis $[0, 2\pi]$
- θ_2: pitch angle relative to x-axis $[0, \pi]$

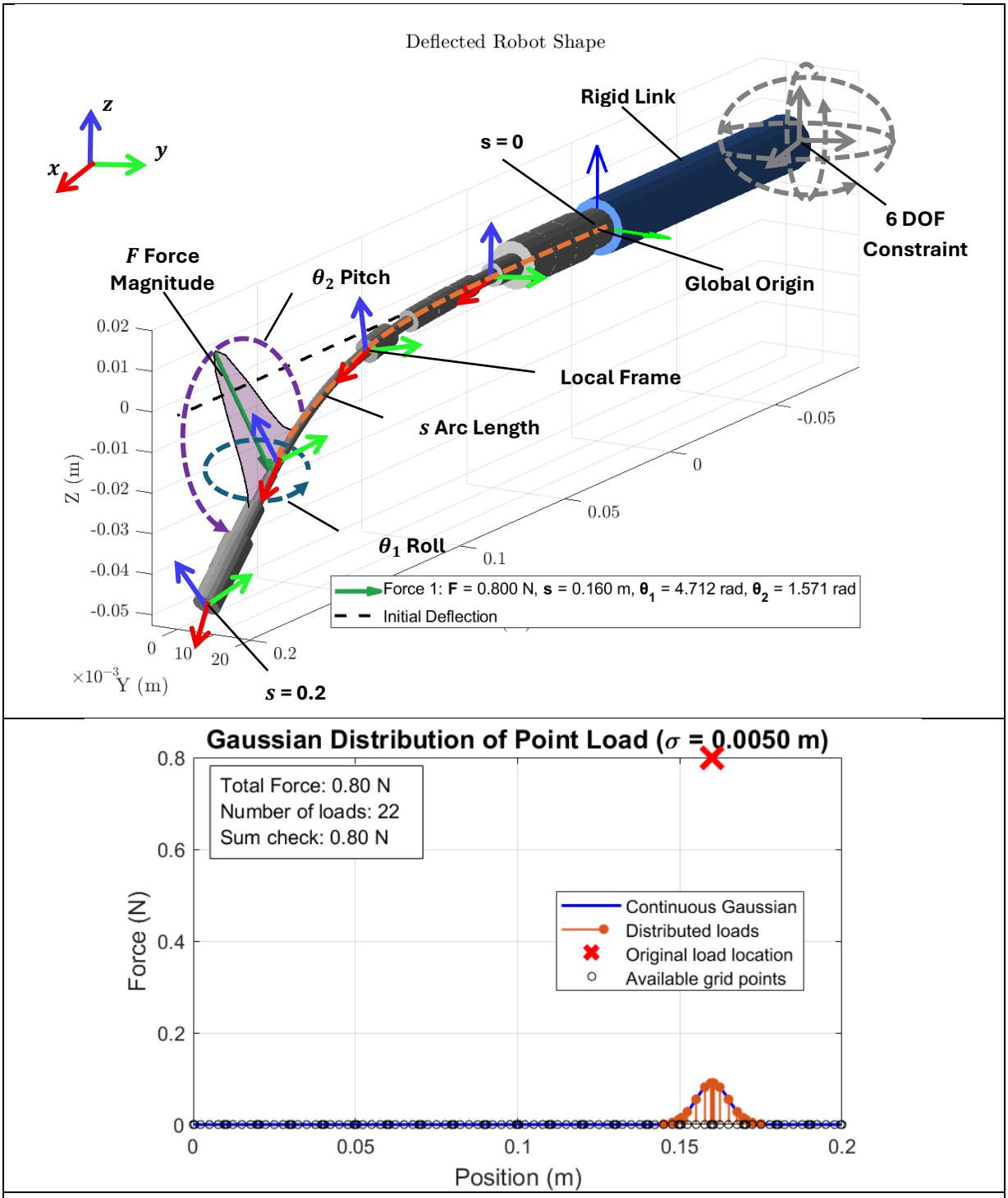

Figure: Schematic showing force application to the robot, force parameterization as a discrete Gaussian distribution, and local reference frames.

These four force components are used to construct a Gaussian distribution that is discretized along the x-axis. The distribution must be discretized because SoRoSim only allows loads to be applied at quadrature points. The number of quadrature points determines the resolution of the applied load distribution; all models in this paper use 100 quadrature points.

The Gaussian distribution is centered at s with a standard deviation of 0.005 by default. The distribution is then scaled by a constant such that the sum of all discrete y-values equals the desired total force magnitude.

Next, the discrete distribution is converted into an array of 3D force vectors. Each vector's magnitude corresponds to a y-value from the distribution, and its orientation is determined by the angles θ_1 and θ_2. A corresponding location array stores the x-coordinate (application position) for each force vector along the backbone.

The applied forces are modeled as follower forces, meaning their orientation is defined relative to the local frame at each application point rather than the global reference frame. Consequently, the angles θ_1 and θ_2 are measured with respect to the backbone at each point of application. Follower forces were chosen to facilitate visualization and reduce computational time for static equilibrium calculations.

Finally, each force vector and its corresponding position are passed to SoRoSim to be applied as a point load. The result is a set of discrete point forces that collectively approximate a Gaussian force distribution along the soft robot backbone.

Experimental Setup

Robot Setup

To compare the force estimation performance of different geometric designs, we built a motorized testing setup to apply two point forces and apply three tendon tensions.

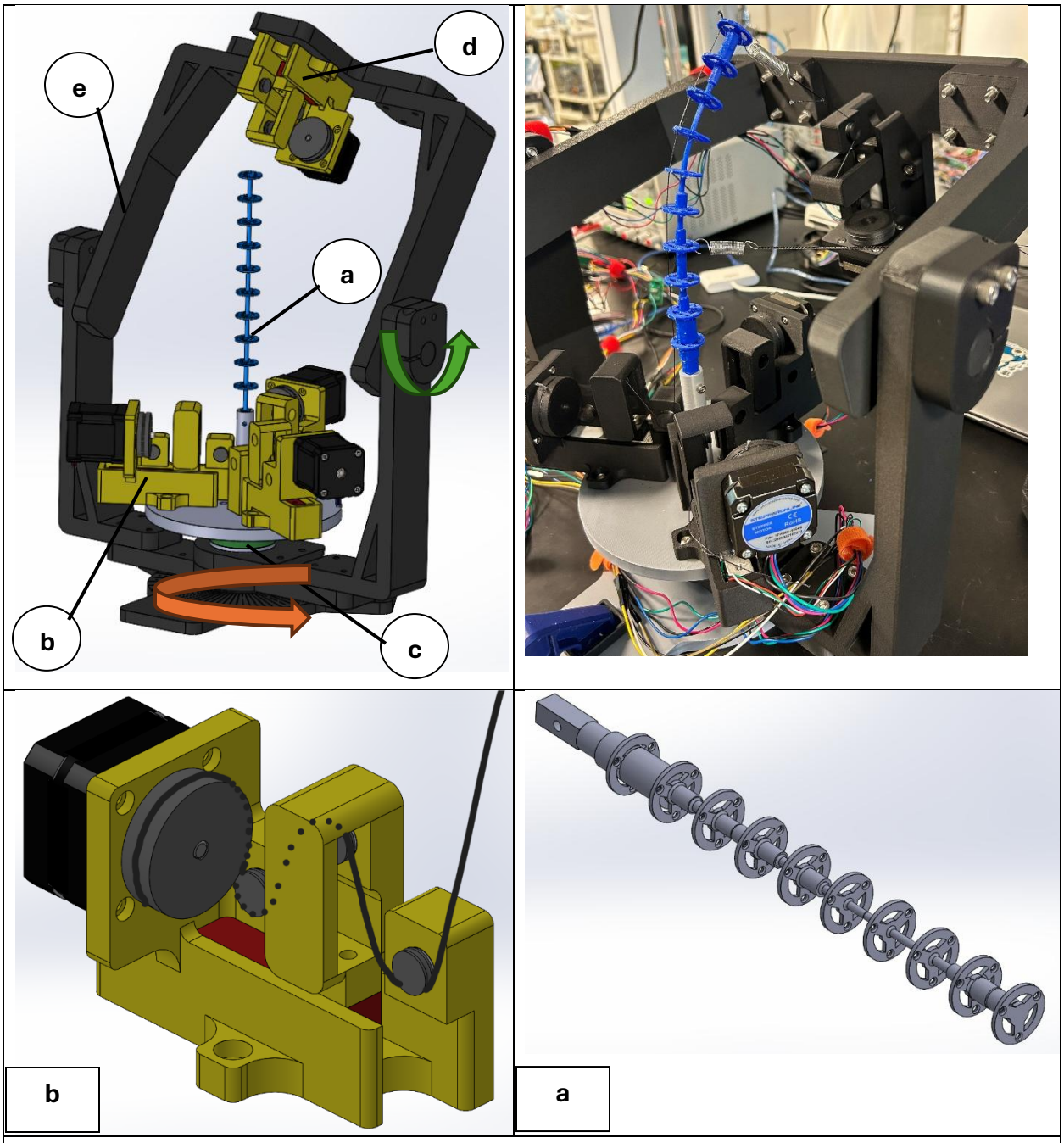

Figure: (a) link design, (b) closed-loop tension-controlled winch for applying tendon tensions, (c) 6-axis force-torque sensor, (d) closed-loop tension-controlled winch for applying point force, (e) adjustable frame with two axes of rotation.

Three winches positioned at 120° intervals around the circumference of the designated beam enable independent closed-loop tension control of each tendon. Each winch uses a NEMA 17 stepper motor and TMC2209 stepper motor driver to wind the tendon in and out.

The tendon wraps around a pulley configuration that directs it through a load cell. When tensioned, the tendon pulls a center piece into the load cell, which measures a force

component proportional to the total tension (the proportion depends on the pulley angles). The system was calibrated using known masses.

The load cell provides high-bandwidth feedback to an Arduino Mega microcontroller running a proportional control algorithm that reduces motor speed as tension approaches the target value.

The adjustable frame provides three degrees of freedom: two rotations and translational adjustment of the force application point by repositioning the hook attachment along the beam's length. An additional winch mounted on the upper frame applies a target external point force. Motor tension targets are set via the Arduino IDE serial interface.

Design of Experiment

For hardware validation we tested two designs:

Design	**Length (mm)**	**Proximal Radius (mm)**	**Distal Radius (mm)**
Cylindrical	200	1.75	1.75
Tapered	200	2.00	1.25
Optimized	200	[1.25, 6]	

For each design, we conducted 40 experimental trials across a range of force inputs. We selected force inputs using JMP's custom design of experiment (DOE), which provides uniform sampling of the input space, identifies the most informative variable combinations, and achieves equivalent accuracy with fewer trials than traditional methods.

The factor levels selected for the experiment are shown below.

Factor name	**# of factor levels**	**Levels**
F_1	10	0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0
s_1	6	0.1, 0.12, 0.14, 0.16, 0.18, 0.2
el_1	5	90°, 135°, 180°, 225°, 270°
F_2	10	0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0
s_2	4	0.2, 0.4, 0.6, 0.8
el_2	3	180°, 225°, 270°
tau_1	8	0, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.4

Forward Model

The forward model is computed using an author-created MATLAB function:

$$W(x,\tau) = get_proximal_value(x,\tau) \in \mathbb{R}^6$$

Input: Applied force $x = [F, s, el, az]$ and tendon tensions $\tau = [\tau_1, \tau_2, \tau_3]$

Output: Resulting proximal wrench $W = [T_x, T_y, T_z, F_x, F_y, F_z]$

Method: SoRoSim statics() function

Computation time: Approximately 0.05 s

Multiple applied forces:

For N applied point forces, stack vertically $x = \begin{bmatrix} F_1, s_1, el_1, az_1 \\ \vdots \\ F_N, s_N, el_N, az_N \end{bmatrix} \mathbb{R}^{Nx4}$

The function takes a user-defined input force and passes it to a custom function that computes the corresponding discretized Gaussian distribution and applies it to the robot. It then runs SoRoSim's statics function to perform a static equilibrium simulation. SoRoSim outputs the proximal wrench required to hold this quasistatic equilibrium configuration under the applied load, and can optionally return the transformation matrix and Jacobian at each significant point. The proximal wrench is then returned to the user. An optional plotting function can then visualize the deflected backbone and tendon shapes along with the applied forces, using the transformation matrices.

Inverse Model

Single Point Force

The inverse model for a single point force is computed using an author-created MATLAB function:

$$\hat{x} = [\hat{F}, \hat{s}, \widehat{el}, \widehat{az}] = get_distal_value(W_m, \tau_m)$$

Input: Measured proximal wrench $W_m = [T_x, T_y, T_z, F_x, F_y, F_z]$ (via ATI Mini40 IP65 6-axis force-torque sensor) and measured tendon tensions $\tau_m = [\tau_1, \tau_2, \tau_3]$ (via load cells)

Output: Estimated external applied force $\hat{x} = [\hat{F}, \hat{s}, \widehat{el}, \widehat{az}]$

Method:
This function finds the force $\hat{x}$ that minimizes the difference between the measured wrench W_m and the simulated wrench W computed using the forward model. The optimal force is

the one that, when passed through the forward model, produces a wrench most closely matching the measurement. We calculate $\hat{x}$ using MATLAB's nonlinear least-squares solver (lsqnonlin) with MultiStart to minimize the weighted residual:

$$\min_{x} \|W(x) - W_m\|^2$$

Upper and lower bounds $l = [0.1, 0.01, 0, 0]$ and $u = [1, 0.2, 2\pi, \pi]$ constrain the four design variables. The weights/normalization vector adjusts the relative importance of each of the six wrench components in the minimization.

We use MultiStart to find multiple local solutions by starting from various initial points x_0, preventing the solver from getting trapped in local minima. This approach provided faster and more accurate convergence than gradient-free methods, like genetic algorithms, in this context. Initial guesses are selected from a Sobol sequence—a low-discrepancy quasi-random sequence that fills the search space more uniformly than random sampling, accelerating convergence compared to Monte Carlo methods. The optimization terminates after testing up to 10 initial guesses or when the residual norm falls below 1×10^{-6}.

Two Point Forces

The inverse model for two point forces is computed using:

$$\hat{x} = \left[\widehat{F_1}, \widehat{s_1}, \widehat{el_1};\ \widehat{F_2}, \widehat{s_2}, \widehat{el_2}\right] = get_distal_value(W_m, \tau_m)$$

Input: Same as single point force model

Output: Estimated two external applied forces $\hat{x} = \left[\widehat{F_1}, \widehat{s_1}, \widehat{el_1};\ \widehat{F_2}, \widehat{s_2}, \widehat{el_2}\right]$

This model assumes both point forces are locally perpendicular to the backbone $\left(az = \frac{pi}{2}\right)$.

Iterative Loading Method

To determine the number of point forces without camera information, we use an iterative approach. The method begins by minimizing the residual norm with a single point force, then iteratively adds forces until no further improvement is observed. This final configuration identifies the optimal number of forces for the loading case.

This approach is particularly effective when forces have significantly different magnitudes. When small and large forces are applied simultaneously, the proximal wrench of the larger force can dominate the signal, causing the optimizer to incorrectly model two equal forces at the larger force location. The iterative method avoids this pitfall by recognizing when a

single force provides a better fit, effectively filtering out negligible smaller forces that would otherwise introduce modeling errors.

Ensuring the inverse model has a unique solution

While it is straightforward to calculate the map from an applied force to proximal wrench, the inverse problem is often ill-posed, especially for two-point forces. To compare how well-posed the inverse mapping is for various robot designs we propose two metrics: local sensitivity and pairwise separation.

Local sensitivity

Local sensitivity measures how much the outputs change for small input perturbations, quantified through Lipschitz continuity. This ensures local injectivity—that nearby wrenches map to distinct point forces, preventing the inverse model from confusing similar inputs. We can measure local injectivity using Lipschitz continuity.

A map $f : \mathbb{R}^n \mapsto \mathbb{R}^m$ is Lipschitz continuous on a domain if there exist constants $L > 0$ such that for all x_1, x_2:

$$L\|x_1 - x_2\| \leq \|f(x_1) - f(x_2)\|$$

If $L > 0$, then f and its inverse are locally injective and stable. The Lipschitz constants L can be approximated by the minimum singular values of the Jacobian over the domain. For the two point force case we would use the 6x6 Jacobian J of the forward map:

$$J(x;r) = \frac{\partial W}{\partial x} = \frac{\partial}{\partial[F_1, s_1, el_1, F_2, s_2, el_2]} \begin{bmatrix} T_x \\ T_y \\ T_z \\ T_x \\ T_y \\ T_z \end{bmatrix}$$

Using the minimum singular value of this Jacobian we have the following relationship:

$$\sigma_{min}(J)\|\Delta x\| \leq \|J\Delta x\|$$

Sampling multiple Jacobians across the entire force domain Ω provides an approximation for our Lipshitz constant L:

$$L \approx \min_{x \in \Omega} \sigma_{min}\big(J(x)\big)$$

If $L > 0$, the forward map is locally injective and stably invertible at the sampled points.

However, this metric does not account for global non-uniqueness (e.g., two far-apart configurations with same wrench)

Pairwise separation (updated 10/06/2025)

Pairwise separation checks that any two different inputs produce clearly different outputs. Imagine plotting every possible input force and its corresponding wrench output. If two very different force inputs give almost the same wrench point, the mapping is not unique — the inverse model can get confused. Pairwise separation measures that gap for every pair of inputs:

$$R_{ij} = \frac{\|W_i - W_j\|}{\|x_i - y_j\|}$$

- If R_{ij} is large, small input changes cause noticeable output changes → good injectivity.
- If R_{ij} is small, two far-apart inputs look almost the same → bad injectivity (ambiguous).

If we minimum R_{ij} across all pairs — this tells us the worst case "confusion" for a given design. We can then optimize the beam geometry so that this minimum ratio is as big as possible.

Optimized Geometry

We now explore abstract geometries and numerical methods for deriving optimal geometries for force sensing. Force estimation accuracy degrades significantly for two-point forces compared to single-point forces. Small deviations in measured wrench—from sensor noise, disturbances, or hysteresis—cause large changes in estimated forces. The optimizer often converges to suboptimal local minima due to an ill-conditioned, nearly singular system. To address this, we propose an optimization framework that improves injectivity through geometric design. We use two metrics: local sensitivity and pairwise separation. Both metrics appear in our objective function to produce designs that are locally stable and globally unique.

Combined objective: The discretized formulation using N sampled inputs is

$$\max_{r \in \mathbb{R}^n} (\alpha \cdot \sigma_{\min} + (1 - \alpha) \cdot \Delta_{\min})$$

where:

- local stability: $\sigma_{\min} = \min_{i=1,\ldots,N} \sigma_{min}(J(x_i; r))$
- global injectivity (pairwise separation: $\Delta_{\min} = \min_{\substack{i,j=1,\ldots,N \\ i \neq j}} \frac{\|W(x_i;r) - W_j(x_j;r)\|}{\|x_i - x_j\|}$

- weighting coefficient: α

subject to:

$$r_{min} \leq r_i \leq r_{max}, \quad i = 1, \dots, M$$

This method uses N sampled inputs $\left[\widetilde{F_1}, \widetilde{s_1}, \widetilde{el_1}, \widetilde{F_2}, \widetilde{s_2}, \widetilde{el_2}\right]$ and M consecutively connected link segments of uniform length with variable radii r_i.

The first term is our local stability metric. It computes the Jacobian of the forward map at each sample point using finite differences, then finds the minimum singular value across all Jacobians. The magnitude $\sigma_{\min}$represents the smallest local gain of the forward map over all directions. Maximizing $\sigma_{\min}$raises the lower bound on input distinguishability, ensuring the mapping remains injective and preventing inverse estimates from exhibiting extreme sensitivity to wrench errors.

The second term is our global stability metric. It compares the norm of input differences to wrench differences for all distinct sample pairs. The minimum ratio $\Delta_{\min}$identifies the worst-case pair most likely to be confused. Maximizing $\Delta_{\min}$raises the lower bound on input separability.

Citations

[1] https://ieeexplore.ieee.org/document/9057619

[2] https://ieeexplore.ieee.org/document/5957337

[3] https://ieeexplore.ieee.org/document/9816130